



DEVELOPER ONBOARDING & SYSTEM DOSSIER

Medyora — India's Smartest AI Healthcare Ecosystem

An enterprise-grade, fullstack digital health platform uniting Patients, Doctors, and Clinics. Featuring real-time OPD queue telemetry, 24/7 Smart Care AI Copilot, instant slot booking, digital prescription generation, and cross-platform native distribution.

CORE ENGINE

**TanStack Start +
Nitro**

FRONTEND STACK

**React 19 & Tailwind
v4**

DATABASE & AUTH

**Supabase
PostgreSQL (RLS)**

MOBILE DISTRIBUTION

**Capacitor Native +
PWA**

AI INTELLIGENCE

**Clinical Knowledge
Engine**

LOCALIZATION

**English, Hindi,
Marathi**

Created & Engineered By

Ritik Kumar & Engineering Team

Binarize Technologies

Project Version: **v2.4 Production Candidate**

Confidentiality: **Internal Engineering Team**

Documentation Date: **September 2026**

Medyora is an all-in-one healthcare super-platform designed to solve India's most critical healthcare delivery bottleneck: **the unorganized, time-consuming Out-Patient Department (OPD) queue and fragmented medical record system**. In traditional clinics and hospitals across India, patients routinely suffer 2 to 3 hours of idle waiting in crowded waiting rooms without real-time queue visibility, while doctors battle handwritten paperwork, scheduling chaos, and lack of historical patient context.

The Healthcare Problem in India

- Unpredictable clinic wait times (120–180 minutes average).
- Illegible, lost, or damaged physical paper prescriptions.
- Absence of verified doctor credentials and pricing transparency.
- Language barriers preventing millions from understanding clinical guidance.
- Inability of clinics to balance walk-in patients with online bookings.

The Medyora Solution

- **Live Digital Token & Queue Tracker:** Know exactly when to arrive.
- **Smart Care AI Copilot:** Instant 24/7 symptom triage & risk assessment.
- **1-Click E-Prescriptions:** Standardized digital Rx with PDF downloads.
- **Multi-Persona Architecture:** Unified hub for Patients, Doctors, and Admins.
- **ABDM & Native Mobile Ready:** Ready for Capacitor iOS/Android & PWA.

Platform Stakeholders & User Personas



Patient & Family

Search specialists by symptoms, book in-clinic/video/home visits, track real-time queue tokens, chat with 24/7 AI copilot, manage family profiles, and store lifelong health records.



Doctor & Clinic Staff

Manage daily clinic OPD, call next tokens with audio cues, author structured digital prescriptions in under 60 seconds, view patient medical dossiers, and review earnings.



Platform Admin / Hospital

Review and verify doctor medical registration credentials (MCI/NMC), audit appointments ledger, manage CMS articles and banners, and govern platform compliance.

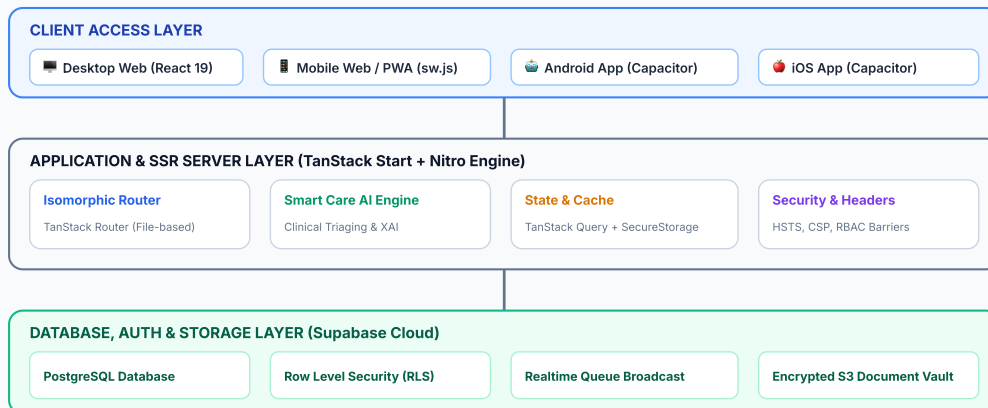
Table of Contents for Onboarding Teammates

SECTION	TITLE	CORE TOPICS COVERED
Sec 02	System Architecture & Tech Stack	TanStack Start (SSR), Nitro engine, React 19, Tailwind v4, Supabase, Capacitor mobile.
Sec 03	Core User Workflows & Flowcharts	Visual journey maps: Patient discovery & booking, Doctor queue & Rx, AI triage pipeline.

SECTION	TITLE	CORE TOPICS COVERED
Sec 04	Modular Directory & Code Breakdown	Deep dive into src/modules/patient, doctor, admin, and shared.
Sec 05	Database Schema & PostgreSQL Models	Tables (profiles, doctors, appointments, prescriptions), RLS policies, Enums.
Sec 06	Security, Auth & Secure Storage	TTL token storage, base64 encryption, CSP policies, RBAC roles (Patient/Doctor/Admin).
Sec 07	Strategic Benefits & Value Drivers	Zero waiting room friction, type safety, multi-lingual support, offline-first PWA.
Sec 08	Limitations & Engineering Trade-offs	Mock vs real database bridges, WebRTC signaling SFU, payment webhooks, compliance.
Sec 09	Scalability & Production Roadmap	Phase 1 to Phase 5: Redis live queue, ABDM / ABHA ID integration, regional voice AI.
Sec 10	Developer Quick-Start Cheatsheet	Setup commands, testing credentials, capacitor mobile compilation, and tips.

Medyora is built using an ultra-modern, high-performance web and native mobile tech stack. It avoids traditional bulky monoliths by leveraging **TanStack Start** for isomorphic Server-Side Rendering (SSR) and streaming, coupled with **React 19**, **Tailwind CSS v4**, and **Supabase**.

SYSTEM ARCHITECTURE TOPOLOGY



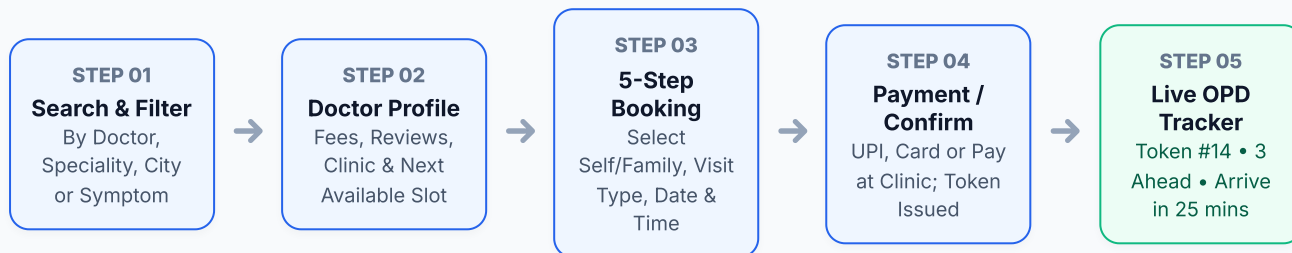
Comprehensive Technology Matrix

LAYER	PACKAGE / TECHNOLOGY	ROLE & ARCHITECTURE PURPOSE
Fullstack SSR	@tanstack/react-start + nitro	Handles server-side rendering, streaming HTML, zero-bundle server functions, and edge readiness.
Routing Engine	@tanstack/react-router	100% type-safe file-based routing in <code>src/routes/</code> with loader hooks and URL state serialization.
UI Library & Primitives	react 19 + @radix-ui/*	Accessible unstyled primitives (Dialog, Tabs, Accordion, Popover, Dropdowns) ensuring WCAG 2.1 compliance.
CSS & Design System	@tailwindcss/vite v4	Next-generation Tailwind CSS v4 engine with instant compilation and universal dark mode support.
Asynchronous State	@tanstack/react-query v5	Server-state management, automated query deduping, background refetching, and optimistic updates.
Backend Database	@supabase/supabase-js	PostgreSQL database integration with Row-Level Security, database functions, and auth sessions.
Mobile Wrappers	@capacitor/cli (iOS & Android)	Compiles the React codebase into native mobile binaries with push notifications, status bar, and camera.

LAYER	PACKAGE / TECHNOLOGY	ROLE & ARCHITECTURE PURPOSE
Animations & Data Viz	framer-motion + recharts	Smooth spring transitions and responsive clinical/revenue charts for doctor and admin portals.
Form Validation	react-hook-form + zod	Declarative type-safe form validation for patient booking, medical registration, and profile setups.
PDF Generation	jspdf	Client-side vector generation of official digital prescriptions with doctor signature blocks.
Localization	i18next + react-i18next	Triple-language support across English, Hindi (हिन्दी), and Marathi (मराठी) across all user modules.

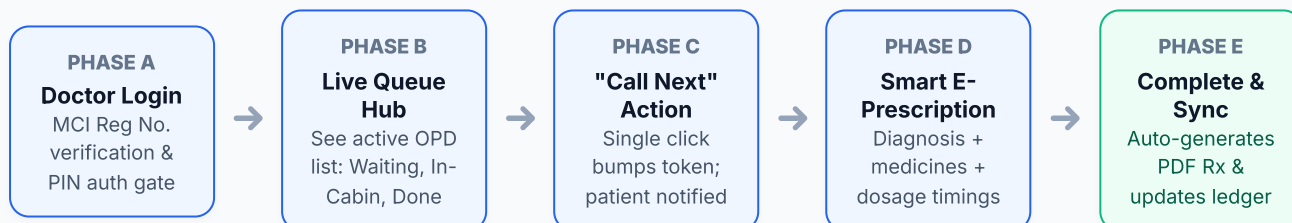
To help any new teammate grasp how Medyora functions at runtime, the platform's four core operational lifecycles are diagrammed below.

WORKFLOW 1: PATIENT DISCOVERY, BOOKING & LIVE TOKEN FLOW



Patient Telemetry Experience: The patient does not need to sit in the waiting room. They track their token on their smartphone from home or a nearby coffee shop. When token #13 enters the cabin, Medyora issues an audio chime and notification alerting the patient to proceed to the cabin.

WORKFLOW 2: DOCTOR OPD QUEUE & PRESCRIPTION AUTHORIZING FLOW



WORKFLOW 3: SMART CARE AI SYMPTOM TRIAGE & RECOMMENDATION ENGINE



WORKFLOW 4: MULTI-ROLE AUTH BARRIER & SECURE STORAGE FLOW



04 Modular Architecture & Code Breakdown

CODEBASE STRUCTURE

The project follows a **Domain-Driven Modular Architecture**. All feature code resides in `src/modules/`, organized cleanly by bounded context. The global routing table in `src/routes/` imports and composes these modular components.

Directory Map: `src/`

```
src/
├── routes/                                # TanStack Start File-based Routing (68 routes)
│   ├── __root.tsx                        # Root HTML Shell, Providers, Command Palette, Meta
│   ├── index.tsx                         # High-converting Landing Page
│   ├── auth.*.tsx                        # Login, Verification, Profile, Language Selection
│   ├── patient.*.tsx                     # Search, Doctor Profiles, Queue, Prescriptions, AI
│   ├── doctor.*.tsx                      # Doctor Dashboard, OPD Queue, Prescriptions, Analytics
│   └── admin.*.tsx                       # Superadmin Platform Governance & Doctor Approvals
├── modules/
│   ├── patient/                          # Patient SuperApp Subsystem (22 domain packages)
│   │   ├── search/                       # Multi-filter Doctor & Clinic Search Page
│   │   ├── appointment/                 # 5-step Booking Engine, Slot Picker & Ledger
│   │   ├── queue/                       # Realtime Live OPD Token Tracker & Audio Chimes
│   │   ├── care-ai/                     # 24/7 Smart Care AI Copilot Engine (270KB+ rules)
│   │   ├── consult/                     # Teleconsultation / WebRTC Encrypted Video Room
│   │   ├── prescriptions/               # Digital Rx Viewer & PDF Exporter (jspdf)
│   │   ├── medical-records/             # Encrypted Health Records & Diagnostic Reports
│   │   ├── family/                      # Multi-profile Family Health Manager
│   │   ├── medicines/                   # Pharmacy, Salt Discovery & Medicine Orders
│   │   └── lab-tests/                    # Diagnostic Packages & Home Sample Collection
│   ├── doctor/                           # Doctor & Clinic Clinical Cockpit (12 domain packages)
│   │   ├── dashboard/                   # Daily OPD Metrics, Appointments & Revenue
│   │   ├── queue/                       # OPD Token Calling, In-Chair & Skip Handlers
│   │   ├── schedule/                    # Slot Timing Config, Working Hours & Vacation
│   │   ├── prescriptions/               # Clinical Rx Authoring with Typeahead Medicines
│   │   └── analytics/                   # Patient Retention, In-clinic vs Video Revenue
│   ├── admin/                            # Governance & Verification Subsystem
│   │   ├── dashboard/                   # Platform GMV, Active Consultations, System Health
│   │   ├── doctors/                     # Medical Council Registration Verification
│   │   └── cms/                          # Article Publishing & Home Banner Control
│   ├── auth/                             # Phone OTP, PIN Gate, and Session Initializers
│   └── shared/                           # Cross-cutting Utilities, Security & Theme System
│       ├── security/                     # secureStorage.ts (TTL Base64), securityHeaders.ts (CSP)
│       ├── auth/                         # useAuth.ts hook, role barriers, custom events
│       ├── theme/                         # ThemeProvider (Dark / Light / System auto-detect)
│       ├── native/                       # Capacitor Native Bridge & Push Notification bindings
│       ├── offline/                       # Offline Banner, Service Worker sync & PWA prompt
│       └── data/                          # Mock stores (mock.ts, doctor-store.ts, superapp-mock.ts)
```

Detailed Feature Matrix by Module

MODULE	COMPONENT / PAGE	CAPABILITY DESCRIPTION
Patient	SearchPage.tsx	Debounced search querying doctor name, specialty, clinic, city, and symptom tags simultaneously with quick specialty

MODULE	COMPONENT / PAGE	CAPABILITY DESCRIPTION
		chips.
Patient	BookingFlow.tsx	Full modal stepper supporting Self vs Family booking, consultation mode selection (Clinic, Video, Home), date/time slot picker, and payment.
Patient	LiveQueue.tsx	Calculates wait time based on (My Token - Current Token) * 15 mins, renders dynamic progress bar, and triggers audio alert on arrival.
Patient	CareAIEngine.ts	Evaluates 15 analysis modes including symptom checking, prescription reader, lab report evaluation, drug interactions, and emergency triage.
Patient	VideoConsultationPage.tsx	Full-screen video call layout with microphone/camera toggling, audio device selection, live messaging, and end-call summary.
Doctor	DoctorLayout.tsx	Secure clinical gate requiring doctor verification (MCI-74892), desktop sidebar navigation, command palette hook, and theme toggle.
Doctor	doctor.queue.tsx	OPD token calling board showing "In the chair", "Waiting" (~12 min clear time), and "Completed", with single-click Next and Skip actions.
Doctor	doctor.prescriptions.tsx	Multi-row medicine authoring (Dosage, Morning/Afternoon/Night, Before/After Food, Duration, Follow-up date) with digital sign-off.
Admin	admin.index.tsx	Platform executive dashboard monitoring doctor onboarding pipeline, patient appointments ledger, and system error rates.

Medyora uses **Supabase (PostgreSQL 14+)** with strict typing auto-generated in `src/integrations/supabase/types.ts`. Every table is guarded by Row-Level Security (RLS) policies and role-checking functions.

Core Database Tables

1. `appointments` — Consultation & Token Ledger

COLUMN	TYPE	DESCRIPTION
<code>id</code>	UUID (PK)	Unique appointment identifier.
<code>patient_id</code>	UUID (FK)	References <code>patient</code> user profile.
<code>doctor_id</code>	UUID (FK)	References <code>doctors.id</code> .
<code>token</code>	INTEGER	Daily OPD sequential queue token (e.g. #14).
<code>appointment_date</code>	DATE	Scheduled visit date.
<code>slot_time</code>	TEXT	Allocated time window (e.g. "10:30 AM").
<code>visit_type</code>	TEXT	"clinic" "video" "home".
<code>status</code>	ENUM	<code>confirmed</code> <code>consulting</code> <code>completed</code> <code>cancelled</code> <code>skipped</code> .
<code>amount</code>	NUMERIC	Consultation fee charged.

2. doctors — Healthcare Specialist Registry

COLUMN	TYPE	DESCRIPTION
id	UUID (PK)	Unique doctor profile identifier.
full_name	TEXT	Doctor's professional name (e.g. Dr. Rajesh Sharma).
speciality	TEXT	Cardiology, Dermatology, Pediatrics, General Medicine, etc.
qualification	TEXT	Degrees (MBBS, MD, DM, FRCS).
experience	INTEGER	Years of clinical practice.
clinic_name / clinic_address	TEXT	Physical clinic branding and geographic location.
fee	NUMERIC	Consultation pricing in INR (₹).
verified	BOOLEAN	Whether medical license credentials have been audited by Admin.
available_today	BOOLEAN	Real-time switch for instant walk-in/OPD queue bookings.

3. prescriptions — Digital Rx Records

COLUMN	TYPE	DESCRIPTION
id	UUID (PK)	Prescription record identifier.
appointment_id	UUID (FK)	Links directly to the completed appointment.
doctor_id / patient_id	UUID (FK)	Parties involved in the medical consultation.
diagnosis	TEXT	Doctor's primary clinical diagnosis and observations.
medicines	JSONB	Array of objects: [{name, dosage, timing, duration}].
advice	TEXT	Dietary precautions, rest, lifestyle suggestions.
follow_up_date	DATE	Recommended return consultation date.

4. profiles & user_roles — Identity & Access Control

User authentication is decoupled into profiles (stores full name, phone number, email, blood group, and emergency contact) and user_roles (stores role mapping patient | doctor | admin). The custom PostgreSQL function `has_role(_user_id, _role)` verifies authorization in microsecond database queries.

Handling medical data demands institutional-grade security. Medyora implements defense-in-depth across the browser, server, and storage layers.



Enterprise Secure Storage (TTL)

Implemented in `src/shared/security/secureStorage.ts`. Serializes sensitive session payloads with Base64 encoding, timestamp tracking, and Time-To-Live (TTL) automatic expiration. Stale or tampered tokens are auto-purged on retrieval.



Enterprise Security Headers (CSP)

Configured in `src/shared/security/securityHeaders.ts` and enforced on every SSR response via `src/server.ts`. Includes strict Content Security Policy, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and HSTS preloading.

Secure Storage Implementation Snippet

```
// src/shared/security/secureStorage.ts
interface StoragePayload<T> {
  value: T;
  createdAt: number;
  expiresAt: number | null;
}

export const secureStorage = {
  setItem<T>(key: string, value: T, ttlSeconds: number | null = null): boolean {
    const payload: StoragePayload<T> = {
      value,
      createdAt: Date.now(),
      expiresAt: ttlSeconds ? Date.now() + ttlSeconds * 1000 : null,
    };
    const serialized = JSON.stringify(payload);
    const encoded = btoa(encodeURIComponent(serialized));
    localStorage.setItem(`sec_${key}`, encoded);
    return true;
  },

  getItem<T>(key: string): T | null {
    const encoded = localStorage.getItem(`sec_${key}`);
    if (!encoded) return null;
    const serialized = decodeURIComponent(atob(encoded));
    const payload: StoragePayload<T> = JSON.parse(serialized);
    if (payload.expiresAt && Date.now() > payload.expiresAt) {
      this.removeItem(key); // Auto-purges expired sessions
      return null;
    }
    return payload.value;
  }
};
```

Role-Based Access Control (RBAC) Workflow

Medyora enforces multi-role authorization at the route layout boundary.

- **Patient Routes (/patient/*):** Accessible to standard users. Handles appointment bookings, family profiles, medical records, and AI triage.
- **Doctor Portal (/doctor/*):** Guarded by `DoctorLayout.tsx`. Inspects `useAuth().isDoctor`. If unauthenticated, displays an interactive clinical verification screen requiring a valid Medical Registration Number (e.g. MCI-74892).
- **Admin Portal (/admin/*):** Guarded by `has_role('admin')`. Grants access to the doctor verification desk, financial audits, and platform settings.

Medyora delivers measurable benefits across the healthcare value chain, directly transforming the patient experience and clinic profitability.



Eliminates 90% of OPD Waiting Anxiety

Patients receive a live token with minute-by-minute estimated arrival times. Instead of waiting 2 hours in a germ-prone waiting room, patients arrive precisely when their token is called.



Paperless E-Prescription Standard

Eliminates medical errors caused by bad handwriting. Prescriptions include dosage schedules (Morning / Afternoon / Night / Meal timing) and export instantly to branded PDF files.



24/7 AI Triage & Copilot

Pre-screens patients using 2,300+ clinical rules, calculating disease probabilities and emergency warnings before booking, ensuring patients consult the right specialist immediately.



Tri-Platform Distribution

Write once, run everywhere: Runs identically on modern web browsers, progressive web app (PWA) with offline support, and native Android/iOS apps via Capacitor.



True Multilingual Reach

Full i18n support in English, Hindi, and Marathi empowers Tier-2 and Tier-3 Indian patients who prefer their mother tongue when navigating medical guidance.



Lifetime Family Health Vault

Families manage elders, spouses, and children under one account, storing lab reports, scans, and past prescriptions securely in the cloud.

Current Limitations & Engineering Trade-offs

To ensure team members know exactly where the project stands today, this section transparently documents current technical boundaries and trade-offs.

AREA	CURRENT IMPLEMENTATION STATUS	RECOMMENDED PRODUCTION FIX
Database Bridge	Supabase types & schema ready, but UI currently relies on rich client stores (mock.ts) for instant local demo capability.	Connect real Supabase client queries across all patient.* and doctor.* pages using useQuery hooks.
Video Teleconsultation	Frontend video room layout with simulated peer states, camera/mic toggling, and in-call messaging is fully built.	Integrate a WebRTC Selective Forwarding Unit (SFU) like LiveKit Cloud or Agora SDK for low-latency multi-peer calls.

AREA	CURRENT IMPLEMENTATION STATUS	RECOMMENDED PRODUCTION FIX
Payment Gateway	Interactive checkout UI with UPI QR, cards, and netbanking flows is built; transaction success is client-confirmed.	Implement server-side webhook signature verification for Razorpay or Stripe in Nitro server functions.
AI Copilot Engine	Runs a 270KB+ deterministic heuristic rule engine; fast and zero-cost, but lacks natural language nuances for rare conditions.	Bridge with an LLM streaming endpoint (e.g. Gemini 2.5 Flash API) with RAG medical document grounding.
Regulatory Compliance	Security headers and TTL storage implemented; official DISHA / ABDM M1-M3 certification is pending.	Complete Indian National Health Authority (NHA) ABDM sandbox testing and implement ABHA ID authentication.

A 5-phase engineering roadmap designed to scale Medyora from a high-performance prototype to a national healthcare infrastructure serving millions of daily consultations.

Phase 1: Real API & Infrastructure Cutover (Immediate Priority)

- **Full Supabase Synchronization:** Replace in-memory mock mutations with live PostgreSQL updates via TanStack Query.
- **Razorpay Payment Gateway:** Integrate server-side order generation and webhook verification for automated doctor fee escrow.
- **LiveKit WebRTC Integration:** Wire up enterprise-grade encrypted video calls with automatic bandwidth adaptation.

Phase 2: Real-time Pub/Sub & High-Concurrency Live Queue

- **Redis / Supabase Realtime Channels:** Move token advancement from polling to WebSocket broadcasts.
- **Sub-Second Chime Alerts:** Ensure immediate synchronization between the doctor's "Call Next" tap and the patient's phone display.
- **SMS & WhatsApp Triggers:** Auto-send WhatsApp alerts (via Twilio/Gupshup) when the patient is 2 tokens away.

Phase 3: ABDM (Ayushman Bharat Digital Mission) Integration

- **ABHA (Ayushman Bharat Health Account) ID:** Enable patients to link and verify their 14-digit national health ID.
- **M1, M2, M3 Compliance:** Implement Health Information Provider (HIP) and Health Information User (HIU) consent managers.
- **National E-Prescription Standard:** Map Medyora digital prescriptions to FHIR / HL7 compliant JSON payloads.

Phase 4: Multi-Modal Voice AI in Regional Indian Languages

- **Voice-First Medical Triage:** Integrate Whisper / Gemini Multimodal models for spoken symptom check in Hindi, Marathi, Tamil, and Telugu.
- **Doctor Voice-to-Rx:** Enable doctors to dictate patient advice; AI transcribes it directly into structured medicine rows and dosages.

Phase 5: Hospital OPD Kiosks & Thermal Printer Hardware

- **Receptionist Kiosk Mode:** Dedicated touch-screen portal for clinic receptionists to quickly issue paper tokens to walk-in patients.
- **Bluetooth Thermal Printers:** Auto-print physical token slips with QR codes that patients can scan to track their token live on their phones.

Welcome to the Medyora codebase! Follow this quick-start guide to spin up the local development environment, test features, and build native binaries.

1. Repository Setup & Launch

```
# 1. Clone the repository and install dependencies
git clone <repo-url> medyora
cd medyora
npm install

# 2. Configure Environment Credentials
cp .env.example .env
# Fill in VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY if testing cloud database

# 3. Start the TanStack Start + Nitro Development Server
npm run dev
# The application will launch locally at http://localhost:3000 (or Vite's assigned port)
```

2. Testing User Roles & Portals Locally

PORTAL	DIRECT URL	TESTING INSTRUCTIONS
Patient Portal	/patient or /	Browse doctors, click "Ask Care AI" , search by symptoms, or book an appointment via /patient/doctor/:doctorId/book.
Live OPD Queue	/patient/queue	Displays Token #14 with live countdown. Simulates automatic queue progression every 8 seconds for demo purposes.
Doctor Portal	/doctor	If prompted with verification barrier, click "Enter Doctor Console (Testing Mode)" to auto-login as Dr. Medical Specialist.
Doctor Queue	/doctor/queue	Click "Call Next" to bump the token counter and observe instant toast notifications.
Prescription Builder	/doctor/prescriptions	Select patient, add medicine rows (Paracetamol, Amoxicillin), specify dosage and duration, and click "Issue Prescription" .
Admin Console	/admin	Review platform KPIs, doctor credential audit logs, and CMS health articles.

3. Mobile Native Compilation (Capacitor)

```
# Build production web bundle
npm run build

# Sync web bundle into native mobile platforms
npx cap sync

# Open Android Studio or Xcode to compile APK / IPA
npx cap open android
npx cap open ios
```

4. Helpful Keyboard Shortcuts & Developer Features

- **Command Palette:** Press `Cmd + K` (or `Ctrl + K` on Windows) anywhere in the application to instantly jump between routes and search doctors.
- **Universal Theme Toggle:** Click the sun/moon icon in the header to switch between Light, Dark, and System mode. All components are styled with Tailwind dark: classes.
- **Multi-Language Switcher:** Switch dynamically between English, Hindi, and Marathi in `/auth/language` or via settings.



You're Ready to Build!

If you have any questions or ideas, reach out to **Ritik Kumar** or the core engineering team. Let's make healthcare accessible, zero-wait, and intelligent for everyone across India!